Sri Lanka Institute of Information Technology

Software Stack Specification

Information Security Project 2026

Project ID – ISP-52

Student Name	Student ID
D.M.M. Pasindu Supushmika	IT23136106
J.T. Mithum	IT23271500

Table of Contents

Table of Contents ii
1. Introduction 1
1.1 Purpose 1
1.2 Intended Audience and Reading Suggestions 1
1.3 Project Scope 1
2. Overall Description 2
2.1 Product Perspective 2
2.2 Product Features 2
2.3 Operating Environment 4
2.4 Design and Implementation Constraints 5
2.5 Assumptions and Dependencies 6
3. System Features 6
3.1 Secure Clipboard Encryption 6
3.2 Secure Paste Authentication 7
3.3 Secure Escalation and Alert System 8
3.4 Clipboard Activity Logging and Monitoring 9
4. External Interface Requirements 10
4.1 User Interfaces 10
4.2 Hardware Interfaces 11
4.3 Software Interfaces 12
4.4 Communications Interfaces 14
4.4.1 Network Communication Protocol 14
4.4.2 API-Based Communication 14
4.4.3 Email Communication Interface 15
4.4.5 Remote Communication and Virtual Clipboard Channels 15
4.4.6 Communication Security Requirements 15
4.4.7 Communication Constraints 15
5. Other Nonfunctional Requirements 16
5.1 Performance Requirements 16
5.2 Safety Requirements 16
5.3 Security Requirements 17
6. References 17

1. Introduction

1.1 Purpose

This document specifies the software requirements for Clipboard Security Manager (CMS) v1.0, a cross-platform background application designed to encrypt and control clipboard data. The CSM intercepts all copy/paste operations. On "Copy" it immediately encrypts the clipboard contents, and on "Paste" it prompts for user authentication before decrypting the data. The goal is to prevent any unauthorized app or user from accessing sensitive clipboard information. This scope includes the entire CSM system (Encryption engine, Authentication module, user dialogs and loggings) but doesn't cover changes to the underlying OS itself. In current mobile an desktop Oses, the clipboard typically has Weak or no access controls. Once the text is copied, any app in the foreground can read it. CSM's design secures this shared resource by applying strong cryptography on the copied data (AES-256) and requiring a password check (Argon2id) on paste, thereby turning the clipboard into a protected vault rather than a public buffer.

1.2 Intended Audience and Reading Suggestions

This report is intended for all stakeholders involved in the CSM project. Developers and System architects will read the full document (especially Section 2 – 4) to understand the detailed functional and interface requirements. Testers will focus on the same sections to identify test cases for each feature. Project managers and product owners should read the introduction (Section 1.1 -1.3) and Overall Description (Section 2) to understand the product's goal, benefits and constraints. Sales and marketing staff can review the Overview (Section 2) and System Features (Section 3) for a high-level feature summary. End users or Domain experts can read Section 1 – 3 to learn what functionality CSM provides. Document writers should read all sections (Especially the Definitions System Features, and Interfaces) to prepare user guides.
The document is organized as follows:

Section 2: Describes the product perspective, key features and environments.
Section 3: Lists detailed system features and functional requirements.
Section 4: Specifies user, hardware, software and communication interfaces.
Section 5: Covers non-functional requirements (performance, safety, security)

As a reading sequence, all audiences should begin with the introduction for context. Then developers/testers proceed to Section 3 – 4 for technical specs, managers start with Section 1 -2 for scope and objectives and documentation teams read through the entire SSS in order.

1.3 Project Scope

Clipboard Security Manager is the software product being specified. Its purpose is to protect sensitive clipboard data in accordance with corporate security goals. By encrypting any copied data and gating paste operations behind strong authentication, CSM prevents the data leakage that researchers have identified as a major privacy risk. For example, studies show that mobile Oss allow all foreground apps to access clipboard content without permission **[1],[2]**, enabling data-stealing attacks. CSM's objectives are therefore to close this gap. Key benefits include:

- **Prevent Data Leakage**: Encrypt clipboard contents on copy so that other apps (or Malware) cannot read them. **[1]**
- **Enforce Authentication:** Require user password entry on paste (verified with Argon2id hashing **[3]**) to ensure only authorized use.
- **Align with Compliance:** Support corporate data – protection policies by securing PII and intellectual property that might otherwise be exfiltrated via copy/paste. **[1] [2]**

- **Use Standard Cryptography:** Leverage industry – standards algorithms (AES – 256 is a NIST – approved block cipher **[4]**, and Argon2id is the 2015 PHC winner designed to resist cracking **[3]**) to meet security requirements.

In summary, CSM v1.0 addresses the company's strategic goal of safeguarding data in user workflows (especially in context like copying passwords or confidential text). Its scope covers the full clipboard – protection subsystem; boarder enterprise goals like central DLP infrastructure or network security are related but outside this SSS.

2. Overall Description

2.1 Product Perspective

Clipboard Security Manager (CSM) is a new, standalone security application rather than a follow-on to an existing product. It runs as a client-side background service on user devices (Windows, Linux, macOS) to enforce encryption and access control on the system clipboard. This product fills a gap left by all major operating systems which is the clipboard is inherently globally accessible and insecure, with minimal built-in access control. **[1] [2]** Unlike existing enterprise DLP or OS components, CSM is designed specifically to sit alongside the OS and intercept copy/paste events for any application.

CSM is not part of a larger software suite. Instead, it integrates with standard OS clipboard APIs to monitor clipboard usage. Conceptually, CSM can be viewed as an "add-on" security layer. It hooks the clipboard subsystem, encrypts data on copy, and unlocks it on paste only after authentication. The functional interfaces include the OS clipboard service (for all copy/paste requests) and the local user interface (for password prompts). In system architecture, CSM operates independently, the OS and other applications need not be modified. For example, research shows clipboard vulnerabilities hash shown that attackers can manipulate or steal shared clipboard data in both local and remote sessions **[1][2][3]**, highlighting the need for a client-side protector like CSM.

A simple block diagram would show CSM as a layer between the user's applications and the OS clipboard. Applications issue copy/paste as usual; CSM transparently encrypts data being placed on the clipboard, and when the user pastes, CSM prompts for a password before decrypting and forwarding the data. No central server or cloud component is required. All encryption and decryption is done locally. In summary, CSM is a self-contained security middleware that augments the existing OS clipboard with strong confidentiality and authentication controls.

2.2 Product Features

Executive Summary:

The Clipboard Security Manager (CSM) runs on standard desktop platforms (Windows, Linux and mac OS) without unusual hardware demands. It supports 64-bit CPUs (x86_64 and ARM64) on modern devices. CSM must integrate with each OS's clipboard API: Win32 on Windows, X11 (and Wayland) on Linux, and NSPasteboard on macOS. Key constraints include user privilege limits (no admin required), sandboxing in Wayland (no global clipboard snooping by design), and cooperation or coexistence with other security agents (DLP, AV). Required software is minimal (no special runtime mandate) and uses common crypto libraries and OS key storage.

Hardware Platforms:

CSM targets desktop and laptop PCs.

- Minimum CPU: 64-bit multi-core (e.g. x86_64 or ARM64).
- Memory: at least ~2 GB; recommended ≥4 GB for comfortable multitasking (specific requirements unspecified).

- Storage: negligible footprint (< 100 MB for application and logs).
- Runs on both consumer and enterprise hardware (modern Intel/AMD CPUs, Apple silicon) with AES instruction support.

Supported Operating Systems:

- Windows 10/11 (x86_64, ARM64): uses the Win32 Clipboard API (Ex: GetClipboardData / SetClipboardData). The clipboard is global per-user, and any process can read it if in foreground (no special OS permission) **[1]**
  (source: Android example; Windows behavior similar).
- Linux (X11): uses the X11 selection mechanism (PRIMARY and CLIPBOARD). Any X11 application can claim and access selections without privileges.
- Linux (Wayland): does not expose a legacy global clipboard to clients. Wayland compositors mediate copy/paste via data-control protocols, isolating applications **[5]**
  (no academic source on Wayland clipboard; see official docs).
- macOS 11+ (Big Sur and later): uses NSPasteboard (generalPasteboard). Clipboard contents can be exchanged between apps freely under standard accessibility permissions (no extra privilege needed) **[1]**
  (source: mobile example; macOS uses similar unrestricted model).

OS	Clipboard API	Access Model	User Permission
Windows	Win32 Clipboard	Global clipboard: any app in foreground can read from the clipboard (no OS prompt) **[1]**	None (runs under user account)
X11/ Linux	X11 Selection	Cut/Copy → CLIPBOARD owner becomes available; any app can GetSelection (no extra permissions)	None
Wayland/ Linux	Wayland Data Control	Copy/Paste handled by compositor; applications cannot spy on other apps' clipboard data by design. **[5]**	May require focus/ activation
macOS	NSPasteboard	Global pasteboard; any app can access clipboard contents via API (no prompt).	None

(Sources for APIs: official OS docs – not academic; global-access nature noted in research.)

Clipboard API Details:
Each OS provides different APIs.

- On Windows, the Win32 API is based on a user interface thread model (calls like GetClipboardData) **[5]**.
- On X11, clipboard is implemented via window properties and selection ownership.
- Wayland's design removes the legacy clipboard: applications must use new data-transfer protocols under the compositor's control.
- macOS offers NSPasteboard for text/objects (official Apple docs describe API). **[1]**

In practice, CSM must handle these differences in an abstract way.

Software Dependencies:

CSM is implemented in Python, ensuring a cross-platform codebase with no proprietary runtime requirement. For cryptographic operations, the system leverages the PyNaCl (binding for libsodium) or cryptography (binding for OpenSSL) libraries to handle AES encryption and Argon2 password hashing. To integrate with OS-level facilities, CSM utilizes cryptography. Hazmat bindings for the Linux Crypto API or PyWin32 for Windows CNG. Key management is handled via the keyring library, providing a unified interface to the OS credential stores, including Windows DPAPI, macOS Keychain, and the Linux Secret Service (via dbus). Where hardware security is available, CSM interfaces with TPM 2.0 modules using the

tpm2-pytss library. For alerting and reporting, the system employs Python's native smtplib for email, pysnmp for network management, or the requests library to transmit data to SIEM APIs via RESTful interfaces.

Security/Environmental Constraints:

CSM runs with normal user privileges. It does not require administrator rights except possibly for auto-update or service installation. On modern OSes, any global clipboard hook must respect sandbox restrictions. For example, on Wayland the agent might need to run with compositor or privileged access, otherwise it cannot intercept other apps' clipboard (Wayland permissions are more stringent than X11). On Windows and macOS, CSM may require accessibility permissions if it needs to simulate UI actions or monitor keystrokes. Copy/paste under remote desktop or virtual machines propagates through RDP/VMware's clipboard channels, these have known attacks, **[5]** so testing in those environments is critical.

Interoperability:

CSM is designed to coexist with other endpoint security agents (antivirus, DLP software, MDM clients) and should use standard APIs. It should not conflict with enterprise DLP agents that also monitor clipboard data; in fact, it can complement them by adding encryption. On managed endpoints, CSM could be deployed via enterprise installers or policies. It does not require cloud services to function.

System Configuration:

- Recommended minimum: Dual-core CPU, 4 GB RAM, latest OS update.
- Target: Quad-core CPU, 8 GB+ RAM for future features (unspecified).
- Deployment modes: typically a per-user background service/daemon (running on user login).
- A machine-wide service (running as SYSTEM) is possible for corporate-wide protection, but not mandatory (unspecified).

Testing and CI:

Testing should cover all supported OS/architecture combinations. Automated CI pipelines must build and test the agent on Windows, Linux (with X11 and Wayland sessions), and macOS. Unit tests should simulate clipboard events; integration tests may require virtual machines or containers for each environment. Continuous Integration is recommended to validate cross-platform compatibility after each change (Ex: using GitHub Actions or Jenkins with appropriate runners/agents).

2.3 Operating Environment

Clipboard Security Manager (CSM) is intended to operate in a modern desktop computing environment where users routinely copy and paste sensitive information between applications. Prior research shows that clipboard contents can be exposed to foreground applications, monitored by malicious software, and carried across local, remote desktop, and virtualized environments, which make the runtime environment an important part of the security design **[1], [2], [3]**.

The software is expected to run on 64-bit desktop and laptop hardware with sufficient memory and storage for a lightweight background service. Since the project relies on standard symmetric encryption and password hashing, the target machines should be modern systems capable of supporting efficient cryptographic processing, including AES acceleration where available **[4] [3]**. CSM is therefore suited to ordinary user workstations rather than specialized high-performance hardware **[4].**

From an OS perspective, the first release is intended for mainstream desktop platforms such as Windows, Linux, and macOS. Exact version numbers may be finalized during implementation and testing, but the software should be compatible with currently supported releases of these operating systems and their clipboard subsystems. This is necessary because clipboard handling differs across environments, and prior studies show that clipboard security issues persist across multiple platforms and session types **[1] [5].**

CSM must coexist peacefully with the software that users normally keep running in the background, including web browsers, office applications, messaging tools, remote desktop clients, virtual machines, and

endpoint security utilities. This is especially important because clipboard traffic often passes between ordinary productivity tools and higher risk environments such as browser-based services, remote sessions, and virtualized desktops **[5]**. The system should also operate alongside security software without interfering with clipboard-related workflows or triggering unnecessary conflicts **[1] [5].**

In practice, the operating environment should allow CSM to run as a lightweight background application with access to the user session clipboard, local encrypted storage, and optional notification capabilities. If email alerts are enabled, the software may also require network connectivity for outbound notification delivery; otherwise, the core protection functions should remain fully functional in an offline local environment **[1] [3] [5].**

2.4 Design and Implementation Constraints

The design of Clipboard Security Manager (CSM) is constrained first by the nature of clipboard systems themselves. Prior research shows that clipboard data can be exposed to foreground applications, monitored by malicious processes, and abused across remote desktop or in-memory attack paths **[1] [2] [5]**. Because of this, CSM must operate as a user-space protective layer around the clipboard rather than assume privileged access to the operating system's internal clipboard mechanisms. The software must intercept copy and paste actions, secure copied content immediately, and enforce user verification without altering the clipboard architecture itself **[1] [5]**.

A second major constraint is cross-platform implementation. The project is intended to run on Windows, Linux, and macOS, but clipboard behavior differs across these operating systems in event handling, background execution, and session isolation. The implementation therefore needs platform-specific adapters or abstraction layers so that the same security workflow can be preserved across all supported environments **[1] [5]**. Clipboard exposure in remote desktop and virtualized environments further increases complexity, because clipboard data may move through host guest or session-based channels beyond the local desktop context **[5]**.

A third constraint is cryptographic correctness and parameter selection. CSM depends on AES-256 for encryption and Argon2 for password hashing, but secure deployment requires correct use of these primitives. NIST's review of AES confirms its continued role as a standard block cipher, while the Argon2 literature shows that implementation quality and parameter selection strongly affect real-world effectiveness **[4] [3]**. The system must therefore avoid weak key handling, minimize plaintext exposure, and ensure that encryption and verification are implemented safely and consistently **[3] [4]**.

A further constraint is performance overhead. Clipboard protection occurs during frequent user actions, so the application must remain responsive when encrypting copied data, prompting for authentication, logging events, and handling alerts. Studies of clipboard usage show that clipboard access is frequent in ordinary workflows, which makes latency and efficiency important design requirements rather than optional optimizations **[1] [5].** The software should therefore use efficient background processing and avoid unnecessary polling, large memory usage, or delayed user feedback.

The software is also constrained by usability and security balance. Since the proposed workflow introduces confirmation prompts and password checks before pasting, the design must prevent abuse without making normal copy-paste tasks unusable. Research on clipboard access shows that blanket blocking is not always sufficient and that user-visible warnings or notifications may be necessary, but these controls still need to remain understandable and minimally disruptive **[1]**. The CSM workflow should therefore trigger security prompts only when required and keep the escalation logic clear to the user **[1] [5].**

There are also constraints related to trust boundaries and data handling. Clipboard attacks may originate locally or through remote-session channels, which means the system must assume the surrounding environment is not fully trusted **[5]**. As a result, sensitive values such as passwords, encrypted clipboard payloads, and audit records must not be exposed in logs, temporary files, or unnecessary memory buffers. This is especially important because a secure algorithm alone does not guarantee security if the implementation leaks data through poor configuration or handling **[3] [4] [5].**

Finally, the implementation is constrained by the scope of the project itself. CSM is a defensive clipboard-protection subsystem, not a full enterprise data-loss-prevention platform. Its implementation must therefore stay focused on clipboard encryption, authentication, alerting, and logging, while broader organizational controls remain outside the system boundary. This keeps the project feasible while still addressing the clipboard security gap demonstrated in the literature **[1] [2] [5]**.

2.5 Assumptions and Dependencies

The CSM assumes that the target desktop operating systems continue to expose clipboard and session-event interfaces that can be observed from a normal user-space background process. This is important because clipboard data can be shared across local apps as well as remote desktop sessions, and remote clipboard transfer is explicitly mediated through clipboard virtual-channel behavior in RDP environments **[1] [5] [8]**. If a platform restricts clipboard interception or session monitoring more aggressively, some protection features may be reduced or unavailable.

The system also assumes the software will run on modern 64-bit desktop hardware with enough CPU and memory to perform frequent encryption and password-verification tasks without noticeable delay. This is a reasonable assumption because AES remains a standard-reviewed block cipher and modern 64-bit processors commonly support AES acceleration, while Argon2id is the mandatory support variant in RFC 9106 and is designed specifically for password hashing and memory-hard protection **[4] [6] [7]**.

A further assumption is that secrets are not stored in plain text and that the deployment environment can support a secure local secret-handling method. This is necessary because classical stored keys are a known attack target, and the cryptographic literature treats permanently stored digital keys as a major weakness unless they are protected or replaced by stronger storage approaches **[9] [10]**. For CSM, this means any key material, salts, or sensitive configuration data should be protected through OS-backed or hardware-backed storage rather than ordinary files.

The project additionally assumes that optional notification features such as email alerts can rely on a valid outbound network path and a functioning message-delivery service. If that dependency is unavailable, the core clipboard-protection workflow should still operate locally, but alerting and remote notification features may need to be disabled or deferred.

In terms of dependencies, CSM depends on the operating system clipboard subsystem, cryptographic libraries implementing AES and Argon2id correctly, and a secure local storage mechanism for configuration and secret material **[6] [7] [8] [9].** If the alerting module is included in the final build, it also depends on a mail or notification service. If the project later reuses components from another implementation, those reused modules must remain compatible with the clipboard security workflow and must not introduce plaintext storage or weak key handling **[9] [10]**.

3. System Features

The Clipboard Security Manager (CSM) provides multiple security-focused features designed to protect clipboard data from unauthorized access, leakage, and misuse. These features are organized based on functional responsibilities within the system.

3.1 Secure Clipboard Encryption

3.1.1 Description and Priority

This feature ensures that all clipboard data is automatically encrypted immediately after a copy operation. It prevents *plaintext exposure of sensitive data in system memory, which is a known vulnerability exploited by malicious applications and clipboard-monitoring attacks* ***[1] [2] [5]***.

- *Priority: High*
- *Benefit: 9*
- *Penalty: 9*
- *Cost: 6*
- *Risk: 5*

3.1.2 Stimulus/Response Sequences

Stimulus 1: User performs a copy operation (Ctrl + C / right-click > Copy)
Response:
- System detects clipboard change
- Extract copied data
- Encrypts data using AES-256
- Replaces plaintext clipboard content with encrypted data.

Stimulus 2: Clipboard content is accessed by another application.
Response:
- Only encrypted data is exposed.
- Plaintext remains inaccessible without decryption.

3.1.3 Functional Requirements

REQ-1: The system shall monitor clipboard events in real time.
REQ-2: The system shall detect copy operations initiated by the user.
REQ-3: The system shall automatically encrypt clipboard data immediately after is it copied.
REQ-4: The system shall replace plaintext clipboard content with encrypted data.
REQ-5: The system shall use a secure encryption algorithm compliant with modern standards (AES-256) **[4]**
REQ-6: The system shall ensure that plaintext clipboard data is not stored after encryption.
REQ-7: The system shall handle encryption failures by preventing clipboard update and notifying the user.

3.2 Secure Paste Authentication

3.2.1 Description and Priority

This feature enforces user authentication before allowing access to encrypted clipboard data. It ensures that only authorized users can decrypt and paste sensitive information, mitigating unauthorized access and misuse [1], [5].

- Priority: High
- Benefit: 9
- Penalty: 9
- Cost: 5
- Risk: 6

3.2.2 Stimulus/Response Sequences

Stimulus 1: User attemps to paste data (Ctrl + V)
Response:
- System intercepts paste action
- Displays confirmation dialog

Simulus 2: User confirms paste action
Response:
- System prompts for password

- User inputs password

Simulus 3: Password entered
Response (Correct Password):
- Decryption clipboard data
- Allow paste operation

Response (Incorrect Password):
- Deny Access
- Trigger security escalation (see Feature 3.3)

3.2.3 Functional Requirements

REQ-8: The system shall intercept paste operations initiated by the user.
REQ-9: The system shall display a confirmation prompt before allowing paste.
REQ-10: The system requires user authentication before decrypting clipboard data.
REQ-11: The system shall verify passwords using a secure hashing algorithm (Argon2) **[3]**
REQ-12: The system shall decrypt clipboard data only after successful authentication.
REQ-13: The system shall deny paste operations if authentication fails.
REQ-14: The system shall prevent unauthorized applications from bypassing authentication.

3.3 Secure Escalation and Alert System

3.3.1 Description and Priority

This feature detects repeated unauthorized access attempts and applies progressive security responses, including alerts and temporary lockouts. It addresses attack scenarios where adversaries attempt brute-force or unauthorized access to clipboard data [5].

- Priority: High
- Benefit: 8
- Penalty: 8
- Cost: 4
- Risk: 6

3.3.2 Stimulus/Response Sequences

Stimulus 1: User enters incorrect password (First Attempt)
Response:
- Display error message
- Allow retry

Stimulus 2: Second failed attempt
Response:
- Send alert (Ex: email notification)
- Allow retry

Stimulus 3: Third failed attempt
Response:
- Lock clipboard access
- Notify user
- Require recovery action

3.3.3 Functional Requirements

REQ-15: The system shall track failed authentication attempts.
REQ-16: The system shall display an error message after the first failed attempt.
REQ-17: The system shall trigger an alert after the second failed attempt.
REQ-18: The system shall lock clipboard access after the third failed attempt.
REQ-19: The system shall require re-authentication or recovery after lockout.
REQ-20: The system shall log all failed access attempts.

3.4 Clipboard Activity Logging and Monitoring

3.4.1 Description and Priority

This feature records clipboard-related events such as copy, paste, and authentication attempts. Logging improves visibility and supports detection of suspicious activity patterns [1], [5].

- Priority: Meddium
- Benefit: 7
- Penalty: 6
- Cost: 3
- Risk: 4

3.4.2 Stimulus/Response Sequences

Stimulus 1: Clipboard operation occurs (Copy/ Paste/ Authentication)
Response:

- System logs event with timestamp

Stimulus 2: Suspicious activity detected
Response:

- System highlights or flag event

3.4.3 Functional Requirements

REQ-21: The system shall log all clipboard operations.
REQ-22: The system shall record timestamps for each event.
REQ-23: The system shall securely store logs.
REQ-24: The system shall prevent logs from containing plain text sensitive data.
REQ-25: The system shall allow authorized viewing of logs.

4. External Interface Requirements

4.1 User Interfaces

The software product should provide a secure, intuitive, and user-friendly graphical user interface (GUI) that enables users to interact efficiently with the system while maintaining strong security and privacy protections. Given the system's focus on clipboard monitoring and data protection, the interface should be designed to clearly communicate security-related events and alerts to users without causing confusion or alert fatigue.

The user interface shall support multiple user roles such as administrators and general users. Each role shall have controlled access to system functionalities based on the principle of least privilege, ensuring that users can only access features relevant to their responsibilities. Sensitive operations shall require additional confirmation or authentication steps.

The interface shall follow consistent design standards across all screens, including uniform color schemes, typography, icons, and layout structures. Security-related visual indicators (such as warnings, alerts, and encryption status) shall be clearly distinguishable using standardized colors (e.g., red for threats, green for secure states, and yellow for warnings). This consistency improves usability and reduces the risk of user error **[11].**

The system shall include the following key interface components:

- Login and Authentication Interface
  Provides secure access using credentials protected with strong hashing mechanisms such as Argon2 [6]. Multi-factor authentication may be integrated for enhanced security.
- Dashboard Interface
  Displays real-time system status, clipboard activity summaries, detected threats, and security alerts. Users shall be able to quickly assess whether their data is secure.
- Clipboard Monitoring Interface
  Shows current and historical clipboard data activity, including detected access attempts by applications. Sensitive data exposure shall be highlighted based on risk levels, aligning with known clipboard attack vectors **[2] [5].**
- Alerts and Notification Interface
  Provides real-time notifications for suspicious activities such as unauthorized clipboard access or data leakage attempts. Alerts shall be clear, actionable, and prioritized based on severity.
- Data Management Interface
  Allows users to view, filter, and manage stored logs of clipboard activities. Search and filtering capabilities shall be provided for efficient analysis.
- Settings and Security Configuration Interface
  Enables users to configure encryption options (e.g., AES-based protection **[4]**), clipboard monitoring sensitivity, access controls, and notification preferences.
- Reports and Analytics Interface
  Presents visual summaries (charts, logs, timelines) of clipboard usage patterns and detected threats to support analysis and decision-making.

All input elements (text fields, dropdowns, checkboxes, etc.) shall include validation mechanisms to prevent incorrect or malicious input. Error messages shall be clear, concise, and non-technical to ensure usability for non-expert users.

The interface shall follow secure design principles to prevent UI-based attacks such as spoofing or clickjacking. Sensitive information displayed on the interface shall be minimized or masked where appropriate, in accordance with secure system design practices **[9].**

The system shall also ensure responsiveness and adaptability across different devices and screen sizes, maintaining usability in both desktop and mobile environments. Important actions such as delete, reset, or overwriting shall require confirmation prompts to prevent accidental data loss.

Keyboard shortcuts may be provided for advanced users to improve efficiency, particularly for administrative tasks. Additionally, a help or guidance feature shall be included to assist users in understanding system functionalities and security alerts.

The following software components require dedicated user interfaces:

- Authentication and login module
- Dashboard and monitoring module
- Clipboard tracking and analysis module
- Alerts and notification system
- Data logging and reporting module
- System configuration and security settings

Detailed screen layouts, mockups, and design specifications shall be documented separately in the User Interface Specification document.

4.2 Hardware Interfaces

The software product shall interact with underlying hardware components through standardized operating system interfaces to ensure efficient, secure, and reliable system performance. Since the system is primarily a software-based security solution, direct hardware interaction is minimal; however, several indirect hardware interfaces are essential for its operation.

The system should support common computing devices, including desktop computers and laptops running modern operating systems (Ex: Windows, Linux, and Android-based systems where applicable). These devices must include standard hardware components such as processors (CPU), memory (RAM), storage devices, and input/output peripherals.

The following hardware interfaces are relevant to the system:

- Processor (CPU) Interface
  The software shall rely on the system's CPU to perform real-time clipboard monitoring, encryption, and threat detection operations. Cryptographic processes such as AES encryption **[4]** and Argon2 hashing **[6]** require computational resources, and the system shall be optimized to minimize performance overhead while maintaining security.
- Memory (RAM) Interface
  Clipboard data is temporarily stored in system memory, making RAM a critical component in the monitoring process. The software should securely access and analyze clipboard content in memory while minimizing exposure to in-memory attacks. This is particularly important given the risks associated with clipboard data leakage and memory-based attacks **[5]**.
- Storage Interface
  The system shall interact with storage devices (HDD/SSD) to store logs, configuration files, and encrypted data. Sensitive information stored on disks shall be protected using secure encryption mechanisms to prevent unauthorized access.
- Input Devices Interface
  The software shall receive input from standard input devices such as keyboards and mice. These inputs are used for authentication, system configuration, and user interaction with the interface. The system shall ensure that sensitive inputs (Ex: passwords) are securely handled and not exposed through clipboard or logging mechanisms.
- Display Interface
  Output shall be presented through standard display hardware, including monitors and mobile screens. The interface shall clearly present system status, alerts, and reports in a readable and structured format.
- Network Interface (Optional/Extended Functionality)
  If the system includes remote monitoring, updates, or cloud-based analytics, it should utilize network

interface hardware (Ex: Ethernet or Wi-Fi adapters). Communication over networks shall be secured using standard protocols such as TLS to prevent interception or tampering of sensitive data.

- Peripheral and External Device Interface
  The system shall account for clipboard interactions involving external devices such as USB drives or remote desktop environments. Clipboard sharing across systems (Ex: via Remote Desktop Protocol) introduces potential vulnerabilities **[8]**, and the software shall monitor and control such interactions where possible.

The communication between software and hardware shall primarily occur through the operating system's application programming interfaces (APIs), ensuring abstraction from low-level hardware details while maintaining compatibility across multiple platforms.
Data exchanged between the software and hardware components shall include:

- Clipboard data retrieved from memory
- User input data from input devices
- Processed and encrypted data stored on storage devices
- System alerts and logs displayed to users

The system shall follow secure design practices to minimize hardware-level vulnerabilities, including limiting unnecessary hardware access, protecting sensitive data in memory, and ensuring secure data transmission between components. These practices align with modern cybersecurity principles for protecting sensitive information in both software and hardware environments **[9], [10].**

In summary, while the software does not directly control hardware at a low level, it relies on standard hardware interfaces provided by the operating system to perform secure clipboard monitoring, data protection, and threat detection functions effectively.

4.3 Software Interfaces

The Clipboard Security Manager shall interface with multiple software components to enable clipboard monitoring, encryption, authentication, logging, and alerting functionalities. These interfaces are designed based on the system's "Capture – Encrypt – Verify – Paste" architecture.

4.3.1 Operating System Interfaces:

The system shall interact with the host operating systems (Windows, Linux, macOS) through native clipboard APIs and system hooks to monitor and control clipboard activity.

- On Windows, the system shall use Win32 Clipboard APIs (Ex: OpenClipboard, GetClipboardData, SetClipboardData)
- On Linux, clipboard interaction shall be handled through X11/Wayland services
- On macOS, the system shall use NSPasteboard APIs

These interfaces enable:

- Detection of copy and paste events
- Retrieval of clipboard data
- Replacement of clipboard content with encrypted data

Clipboard interaction is event-driven and occurs in real time, which is critical due to known clipboard vulnerabilities **[2] [5]**.

4.3.2 Cryptographic Module Interface:

The system shall use Python cryptographic libraries to implement secure encryption and hashing.

- Encryption: AES-256-GCM using cryptography library (v41+)
- Hashing: Argon2 using argon2-cffi library (v23+)

Data flow:

- Input: Plain-text clipboard data
- Output: Encrypted ciphertext stored in clipboard

This ensures clipboard data remains protected while stored in memory, aligning with secure data-at-rest practices **[4] [6]**.

4.3.3 Authentication Interface:
The system should include an internal authentication module implemented in Python.
- User password input shall be hashed using Argon2
- The system shall compare input hash with stored hash

Data exchange:
- Input: User-entered password
- Output: Authentication result (success/failure)

This module enforces secure access control before allowing clipboard decryption.

4.3.4 Database Interface (Local Storage):
The system shall use SQLite (via Python sqlite3 module) for local data storage.
Stored data includes:
- Authentication attempt counters
- Security logs
- Configuration settings

Constraints:
- Sensitive data shall be encrypted before storage
- Logs shall follow a 30-day rolling retention policy

The database shall be accessed using SQL queries through Python's built-in database interface.

4.3.5 Email Notification Interface:
The system shall interface with email services to send alerts during security escalation.
Two possible implementations:
- SMTP using Python smtplib (standard library)
- OR third-party APIs (SendGrid / Mailgun via HTTP requests using *requests* library)

Data exchange:
- Recipient email
- Alert message
- Timestamp

Triggered during:
- Second failed authentication attempt
- Third failed attempt (account lockout)

4.3.6 Logging and Monitoring Interface:
The system shall use Python's logging module to record system events.
Logged events include:
- Clipboard access attempts
- Authentication failures
- Security alerts

Logs shall be stored securely and optionally written to the SQLite database.

4.3.7 Data Exchange Between Components:
The following data shall be shared across modules:

Data Type	Source	Destination	Purpose
Clipboard Data	OS interface	Encryption Module	Secure data
Encrypted Data	Encryption Module	Clipboard	Protect memory
User Input	UI	Auth Module	Verify identity
Auth Result	Auth Module	System	Allow/Deny paste
Alert Data	System	Email Module	Notify user
Logs	System	Database	Audit tracking

4.3.8 Communication Mechanisms

- Internal communication:
  - Python function calls
  - Object/data structure passing (dictionaries, classes)
- External communication:
  - REST APIs using HTTP/HTTPS (requests library)
  - SMTP for email alerts
- Database communication:
  - SQL queries via sqlite3 module

4.3.9 Implementation Constraints

The system shall not use global shared memory unless required by OS clipboard handling
Clipboard data shall always be stored in encrypted form
Sensitive data (passwords, clipboard content) shall never be stored in plaintext
The system shall maintain cross-platform compatibility across Windows, Linux, and macOS
Real-time clipboard monitoring shall be implemented efficiently to avoid high CPU usage

4.4 Communications Interfaces

The Clipboard Security Manager shall implement secure, minimal, and event-driven communication mechanisms to support alert notifications, optional external services, and system integrity. Given the sensitive nature of clipboard data and known attack vectors, all communication interfaces shall be carefully controlled to prevent data leakage and unauthorized access.

4.4.1 Network Communication Protocol

The system shall utilize standardized communication protocols for all external interactions:

- HTTPS (Hypertext Transfer Protocol Secure)
  Used for:
  - Communication with external APIs (Ex: email services)
  - Secure transmission of alert-related data
- SMTP (Simple Mail Transfer Protocol)
  Used for:
  - Sending automated security alert emails

All communications shall be secured using Transport Layer Security (TLS) to ensure confidentiality, integrity, and authenticity of transmitted data. This is critical due to the risk of interception and data exposure in unsecured communication channels **[9].**

4.4.2 API-Based Communication

The system shall communicate with external services using RESTful APIs implemented via Python.

- Communication library: Python requests
- Data format: **JSON (JavaScript Object Notation)**

Each API request shall include:

- Event type (e.g., failed authentication, account lockout)
- Timestamp
- System identifier (non-sensitive)
- Recipient details (for alerts)

No sensitive clipboard data or authentication credentials shall be transmitted through these interfaces.

4.4.3 Email Communication Interface

The system shall generate automated email notifications as part of its **security escalation protocol**.
Trigger conditions:
- Second failed authentication attempt
- Third failed attempt resulting in account lockout

Email structure:
- Subject: Security Alert – Unauthorized Clipboard Access
- Body: Includes event description, timestamp, and warning message

Email transmission shall be performed using:
- Python smtplib (SMTP protocol)
- OR secure third-party APIs (Ex: SendGrid, Mailgun)

This mechanism ensures timely user awareness of suspicious activities while maintaining confidentiality.

4.4.4 Web-Based Communication (User Interaction Context)

The system shall consider indirect communication through web browsers, particularly when users paste data into external web applications such as AI chat systems.
Clipboard data exposure via web interfaces has been identified as a major security risk, especially when users unknowingly paste sensitive data into untrusted environments **[1] [2].**
To mitigate this:
- The system shall enforce authentication before allowing paste operations
- Unauthorized paste attempts shall be blocked

This ensures that sensitive data is not transmitted unintentionally through web-based platforms.

4.4.5 Remote Communication and Virtual Clipboard Channels

The system shall account for communication over remote desktop environments.
- Clipboard data may be transmitted through Remote Desktop Protocol (RDP) virtual channels
- This introduces risks of remote clipboard interception and data leakage **[5] [8]**

The system shall:
- Monitor clipboard activity in remote sessions
- Ensure only encrypted data is available in shared buffers

4.4.6 Communication Security Requirements

All communication interfaces shall adhere to strict security requirements:
- Use of TLS encryption for all external communications
- No transmission of plaintext sensitive data (clipboard content, passwords)
- Secure API authentication where applicable
- Protection against:
    - Man-in-the-Middle (MITM) attacks
    - Replay attacks
    - Data interception

Additionally, encryption standards such as AES are used internally to protect data before any interaction occurs **[4].**

4.4.7 Communication Constraints

The system shall function fully offline for core operations:

- Clipboard encryption
- Authentication
- Logging

Internet connectivity is only required for:

- Email alerts
- External API communication

Failure of communication services shall not affect core functionality

5. Other Nonfunctional Requirements

5.1 Performance Requirements

The Clipboard Security Manager is designed as a lightweight, real-time background application that ensures strong security without compromising system performance or user experience. The system shall perform encryption of clipboard data immediately after a copy event is detected, with a maximum acceptable latency of approximately 100 milliseconds under normal operating conditions. This requirement ensures that the encryption process is effectively invisible to the user while still securing sensitive data in memory. Similarly, the decryption and authentication process during paste operations, including password verification using Argon2, shall be completed within 200 milliseconds to maintain a seamless interaction flow while preserving resistance to brute-force attacks **[6]**.

To ensure efficiency, the system shall maintain low resource consumption, with CPU utilization remaining within 5–10% during active monitoring and memory usage kept below 150 MB under typical workloads. Clipboard monitoring shall operate continuously in real time, ensuring that no clipboard events are missed. This is particularly important given the rapid exploitation window associated with clipboard-based attacks, where malicious processes can access data immediately after it is copied **[2] [5]**. The use of AES-256-GCM encryption further supports performance requirements by providing a well-established balance between computational efficiency and cryptographic strength **[4].**

Additionally, logging and data storage mechanisms shall be optimized to minimize disk input/output operations, especially considering the implementation of a 30 day rolling log policy. The system shall also support multiple consecutive clipboard operations without performance degradation, ensuring stability even under frequent user interaction.

5.2 Safety Requirements

The Clipboard Security Manager shall ensure that its operation does not result in unintended data loss, system instability, or disruption to normal user workflows. Since the application actively modifies clipboard behavior, it is critical that clipboard data always remains accessible and recoverable. In the event of a system failure, crash, or unexpected termination, the system shall fail in a safe manner, ensuring that clipboard data does not remain permanently encrypted or inaccessible to the user.

The application shall preserve the integrity of clipboard contents throughout the encryption and decryption processes, avoiding any corruption or unintended modification of user data. Furthermore, the system shall not interfere with essential operating system functions or other running applications. All security-related interactions, such as authentication prompts and confirmation dialogs, shall be designed to prevent accidental user actions that could lead to data exposure, while still allowing users to maintain control over their workflow.

Error handling mechanisms shall be implemented to ensure that any faults are managed gracefully, with clear and understandable feedback provided to the user. Additionally, the system shall implement controlled responses to authentication failures, such as warnings and temporary lockouts, rather than abrupt

termination or data deletion. These design considerations align with established usability and safety principles, ensuring that the system minimizes human error and operates reliably under all conditions **[11].**

5.3 Security Requirements

Security is the core objective of the Clipboard Security Manager, and the system shall enforce comprehensive measures to protect sensitive data from unauthorized access, leakage, and manipulation. All clipboard data shall be encrypted immediately upon capture using AES-256-GCM, ensuring that data stored in system memory is never exposed in plaintext form. This approach addresses the inherent vulnerability of clipboard architectures, where data is typically accessible to any application with basic permissions **[4]**. As a result, even if malicious software attempts to access the clipboard, it will retrieve only encrypted ciphertext.

User authentication shall be required before any encrypted data can be decrypted and pasted. Passwords shall be securely processed using Argon2 hashing, which provides strong resistance against brute-force and side-channel attacks due to its memory-hard design **[6]**. The system shall implement a multi-level security escalation protocol to further enhance protection. A first failed authentication attempt shall result in a warning message, a second failed attempt shall trigger an automated email alert, and a third failed attempt shall result in a temporary account lockout. This layered approach ensures both prevention and detection of unauthorized access attempts.

All external communications, including email notifications and API interactions, shall be secured using protocols such as HTTPS and SMTP over TLS to prevent interception and tampering. Sensitive data, including clipboard content and authentication credentials, shall never be transmitted over these channels. Instead, only minimal metadata, such as timestamps and event descriptions, shall be communicated.

The system shall also be designed to mitigate known clipboard-related threats, including clipboard hijacking, unauthorized monitoring by background applications, and remote data leakage through protocols such as Remote Desktop Protocol (RDP) **[2] [5] [8]**. Furthermore, the system shall address the growing risk of unintentional data exposure through web-based platforms, such as AI applications, by enforcing authentication before paste operations **[1]**.

To support accountability and forensic analysis, the system shall maintain encrypted logs of all security-relevant events, including authentication attempts and clipboard access activities. These logs shall follow a 30-day rolling retention policy and shall be protected against unauthorized access. The overall system design shall adhere to fundamental cybersecurity principles, including confidentiality, integrity, and availability, while also following secure-by-design practices and minimizing reliance on exposed secrets **[9] [10]**.

6. References

[1] Y. Chen, R. Tang, C. Zuo, X. Zhang, L. Xue, X. Luo, and Q. Zhao, "Attention! Your Copied Data is Under Monitoring: A Systematic Study of Clipboard Usage in Android Apps," in Proc. 46th IEEE/ACM Int. Conf. Softw. Eng. (ICSE), Lisbon, Portugal, 2024, pp. 1–13.

[2] X. Zhang and W. Du, "Attacks on Android Clipboard," Syracuse University, Syracuse, NY, USA, 2013.

[3] P. Tippe and M. P. Berner, "Evaluating Argon2 Adoption and Effectiveness in Real-World Software," arXiv preprint arXiv:2504.17121, Apr. 2025.

[4] N. Mouha, "Review of the Advanced Encryption Standard," Nat. Inst. Stand. Technol. (NIST), Gaithersburg, MD, USA, NISTIR 8319, Jul. 2021.

[5] G. Ammar, A. Okolica, and G. L. Peterson, "Remote Clipboard Data In-Memory Attacks and Detection," in Proc. 3rd Conf. Cybersecurity Educat., Res. Practice (CCERP), Kennesaw, GA, USA, 2024, pp. 1–16.

[6] A. Biryukov, D. Dinu, D. Khovratovich, and S. Josefsson, "Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications," RFC 9106, Sep. 2021.

[7] N. Mouha, "Review of the Advanced Encryption Standard," National Institute of Standards and Technology (NIST), Gaithersburg, MD, USA, NISTIR 8319, Jul. 2021.

[8] Remote Desktop Protocol: Clipboard Virtual Channel Extension, MS-RDPECLIP, Microsoft Corporation, Redmond, WA, USA.

[9] U. Rührmair, "Secret-free security: A survey and tutorial," Journal of Cryptographic Engineering, vol. 12, Mar. 2022.

[10] M. H. Murtaza, H. Tahir, S. Tahir, Z. A. Alizai, Q. Riaz, and M. Hussain, "A portable hardware security module and cryptographic key generator," Journal of Information Security and Applications, vol. 70, p. 103332, Nov. 2022.

[11] A. Dix, J. Finlay, G. D. Abowd, and R. Beale, Human-Computer Interaction, 3rd ed. Harlow, U.K.: Pearson, 2004.

Important guidelines for writing the SSS:

- Remember that you are writing a professional as well as a technical document. So do not make it a "fancy document".
- Use "Times New Roman" font. Main text (body text) must be in 12 pts. Use suitable sizes for headings (You can use the default sizes given in the SSS_Template.docx". Line spacing for main text must be 1.5.
- All pages except the cover page must be numbered, and the number must appear on the lower right-hand corner of the page.
- Number all figures and data table and refer to them in the main text using the numbers.
- Maximum number of pages must be 50.

NOTE :

- Following the guidelines, templates and standards are important, note that marks will get deducted otherwise.